

UNIJUÍ — Universidade Regional do Noroeste do Estado do Rio Grande do Sul  
Linguagens Formais e Compiladores (GRP00543) — 2026/2º  
Prof. Marcos Ronaldo Melo Cavalheiro

# **Gambiarra → C**

Trabalho Prático 1 — Relatório Técnico de Especificação  
Projeto e implementação de uma linguagem de programação temática e de um  
transpilador

Eduardo Manhabosco

Rafael

Felipe

Gabriel

Ijuí, 25/09/2026

# 1. Tema e objetivos

**Gambiarra** é uma linguagem pequena com o vocabulário do "concerto improvisado": o programa é uma gambiarra, cada variável é um remendo, e os valores lógicos são firme (verdadeiro) e quebrado (falso). O transpilador, escrito em Python, traduz um programa .gam para C (a linguagem destino de referência do enunciado), que depois é compilado com o gcc.

**Objetivos:** formalizar o léxico (alfabeto  $\Sigma$ , tokens e expressões regulares) e a sintaxe (GLC em EBNF); implementar à mão o analisador léxico e o parser por descida recursiva, com construção da AST; fazer uma análise semântica com tabela de símbolos, escopo e tipos; e gerar C que preserva o comportamento do programa.

```
// Teste 01: declaração, atribuição e saída (enunciado §8).
gambiarra {
    remendo fita = 10;           // inteiro (deduzido do valor)
    remendo cola = 2.5;         // real
    remendo ligado = firme;     // lógico

    fita = fita + 5;
    cola = cola * 2;

    mostra("fita: ", fita);
    mostra("cola: ", cola);
    mostra("ligado: ", ligado);
    mostra("Sistema Gambiarra iniciado");
}
/* Saída esperada do programa:
fita: 15
cola: 5
ligado: firme
Sistema Gambiarra iniciado
*/
```

## 1.1 Comandos da linguagem

| Gambiarra                           | Significado                               | C gerado                         |
|-------------------------------------|-------------------------------------------|----------------------------------|
| gambiarra { ... }                   | início e fim do programa                  | int main(void) { ... return 0; } |
| remendo x = valor;                  | declaração; o tipo vem do valor           | int / float / int (lógico)       |
| x = valor;                          | atribuição                                | v_x = ...;                       |
| mostra(a, b, ...);                  | saída de vários itens; pula linha no fim  | printf("...\n", ...);            |
| pega(x);                            | entrada de inteiro ou real                | scanf("%d" / "%f", &v_x);        |
| se (c) { } senao { }                | decisão com alternativa                   | if (c) { } else { }              |
| enquanto (c) { }                    | repetição controlada por expressão lógica | while (c) { }                    |
| firme / quebrado                    | verdadeiro / falso                        | 1 / 0                            |
| + - * / %, == != > >= < <=, &&    ! | operadores                                | os mesmos                        |
| // ... e /* ... */                  | comentários                               | —                                |

Escopo mínimo do enunciado (§1): três tipos primitivos (inteiro, real, lógico); declaração e atribuição; `+` `-` `*` `/` com precedência e parênteses; operadores relacionais; `se/senao`; `enquanto`; `pega` e `mostra`; programa e blocos delimitados por `gambiarra { }` e `{ }`.

## 2. Três decisões próprias de projeto

**1. Declaração sem tipo escrito, com tipo deduzido do valor.** Na BIRL-Lite o tipo vem antes do nome (`MONSTRO x = 3;`) e o valor é opcional. Na Gambiarra escreve-se `remendo x = 3;`; o valor é **obrigatório** e a variável recebe o tipo dele (`3`  $\rightarrow$  inteiro, `2.5`  $\rightarrow$  real, `firme`  $\rightarrow$  lógico). A tipagem continua **estática**: o tipo é fixado na declaração e conferido pela semântica (regras R7 e R12). *Justificativa*: há menos palavras reservadas, nenhuma variável existe sem valor (no C não sobra variável sem inicialização) e o tipo nunca contradiz o valor inicial.

**2. Blocos delimitados por chaves obrigatórias.** A BIRL-Lite fecha programa, *if* e *while* com a mesma palavra (`BIRL`). Na Gambiarra, todo bloco (programa, `se`, `senao`, `enquanto`) é delimitado por `{ }`, sempre. *Justificativa*: fica visível onde cada bloco começa e termina, e o fechamento explícito elimina a ambiguidade do "else pendurado": cada `senao` vem logo depois do `}` do seu `se`.

**3. Saída com vários itens e quebra de linha automática.** A BIRL-Lite mostra um item por comando e exige `\n` no texto. Na Gambiarra, `mostra("total: ", x, " de ", y);` mistura textos e expressões separados por vírgula, e cada `mostra` termina a linha. *Justificativa*: ninguém esquece o `\n`, e o gerador monta um único `printf`, escolhendo o formato de cada item pelo tipo calculado na semântica (`%d`, `%g`, `firme/quebrado`).

Outras diferenças em relação à BIRL-Lite: tipo lógico, operadores lógicos `&&` `||` `!` na gramática, operador `%`, menos unário e comentários de linha e de bloco.

## 3. Especificação léxica

### 3.1 Alfabeto $\Sigma$

$\Sigma = \{ a-z, A-Z \text{ (sem acento)} \} \cup \{ 0-9 \} \cup \{ \_ " . , ; ( ) \{ \} + - * / \% = ! < > \& | \backslash \} \cup \{ \text{espaço, tabulação, quebra de linha} \}$ . Dentro de um texto ou de um comentário, qualquer caractere é aceito (inclusive acentos). Um programa é uma palavra de  $\Sigma^*$ . Cada categoria de token é uma **linguagem regular** sobre  $\Sigma$ , descrita por uma expressão regular (Menezes, 2011, cap. 3), e o léxico reconhece essas palavras dentro do programa. Um símbolo fora de  $\Sigma$  (ex.: `@`, `#`, `ç`) é erro léxico.

### 3.2 Tabela de tokens

Notação das ER: a do Python/JFlex (`|` = união, `*` = zero ou mais, `+` = um ou mais, `[]` = classe de caracteres). Nos slides da Aula 3, a união aparece como `+`.

| Categoria         | Token                                                 | ER / padrão       | Lexema de exemplo | Descrição                                                |
|-------------------|-------------------------------------------------------|-------------------|-------------------|----------------------------------------------------------|
| palavra reservada | GAMBIARRA, REMENDO, MOSTRA, PEGA, SE, SENAO, ENQUANTO | a própria palavra | remendo           | programa, declaração, saída, entrada, decisão, repetição |
| literal lógico    | FIRME, QUEBRADO                                       | firme   quebrado  | firme             | verdadeiro / falso                                       |

|                |                                                                          |                                          |                         |                                                            |
|----------------|--------------------------------------------------------------------------|------------------------------------------|-------------------------|------------------------------------------------------------|
| identificador  | ID                                                                       | <code>[a-zA-Z_][a-zA-Z0-9_]*</code>      | <code>fita_2</code>     | nome de variável; diferencia maiúsculas                    |
| número inteiro | NUM_INT                                                                  | <code>[0-9]+</code>                      | <code>42</code>         | inteiro sem sinal                                          |
| número real    | NUM_REAL                                                                 | <code>[0-9]+\.[0-9]+</code>              | <code>10.5</code>       | exatamente um ponto, com dígitos dos dois lados            |
| texto          | TEXTO                                                                    | <code>"(\^"\\n   \\[nt"\\])"</code>      | <code>"oi\n"</code>     | só no mostra; escapes <code>\n \t \" \\</code>             |
| aritmético     | MAIS, MENOS, VEZES, DIVIDE, RESTO                                        | <code>+ - * / %</code>                   | <code>*</code>          | operações aritméticas                                      |
| atribuição     | ATRIB                                                                    | <code>=</code>                           | <code>=</code>          | guarda um valor                                            |
| relacional     | IGUAL_IGUAL, DIFERENTE, MAIOR, MAIOR_IGUAL, MENOR, MENOR_IGUAL           | <code>== != &gt; &gt;= &lt; &lt;=</code> | <code>&gt;=</code>      | comparações                                                |
| lógico         | E_LOGICO, OU_LOGICO, NAO_LOGICO                                          | <code>&amp;&amp;    !</code>             | <code>&amp;&amp;</code> | e, ou, não                                                 |
| delimitador    | ABRE_PAREN, FECHA_PAREN, ABRE_CHAVE, FECHA_CHAVE, PONTO_VIRGULA, VIRGULA | <code>( ) { } ; ,</code>                 | <code>{</code>          | agrupamento, blocos e separação                            |
| ignorado       | —                                                                        | <code>[ \t\r\n]+</code>                  |                         | espaços e quebras de linha                                 |
| ignorado       | —                                                                        | <code>//[^\n]*</code>                    | <code>// nota</code>    | comentário de linha                                        |
| ignorado       | —                                                                        | <code>/\*[^\s\S]*? \*/</code>            | <code>/* ... */</code>  | comentário de bloco (termina no primeiro <code>*/</code> ) |

### 3.3 Como o léxico resolve os conflitos

O léxico (`lexico.py`) tem uma tabela de regras (*nome, expressão regular*), testada **em ordem** a cada posição do texto. A primeira que casa ganha, e o lexema vira token (ou é ignorado, ou vira erro).

- **Palavra reservada × identificador:** se casa com a ER de ID; depois o léxico consulta a tabela `PALAVRAS_RESERVADAS` e, se o lexema está nela, o token vira `SE`. Por isso `sera` continua sendo identificador.
- **Maior casamento (longest match):** `== != > >= < <= && ||` vêm antes de `= ! > <`. Sem isso, `==` viraria dois tokens `=`. `NUM_REAL` também vem antes de `NUM_INT`.
- **Fim do número:** `NUM_INT` e `NUM_REAL` exigem que o número acabe ali (`(?![\w.])`). Sem isso, `2abc` viraria `NUM(2)` `ID(abc)` e `3.4.5` viraria `NUM(3.4)` + resto.
- **Posição:** cada token guarda linha e coluna. A cada lexema, a coluna avança o tamanho dele; se houver quebra de linha, a linha avança e a coluna recomeça.
- **Erros:** regras de erro (número malformado, identificador com acento, texto sem fechar, escape inválido, comentário sem fechar e o "pega-tudo" `.`) registram **linha, coluna e lexema** e a leitura **continua**. Se houver erro léxico, o parser não roda.

## 4. Especificação sintática: $G = (V, T, P, S)$

$V = \{ \text{Programa, Bloco, Instrucao, Declaracao, Atribuicao, Leitura, Escrita, Item, Condicional, Repeticao, Expressao, ExprE, ExprRel, OpRel, ExprAdit, Termo, Unario, Fator} \}$

$T = \{ \text{gambiarra, remendo, mostra, pega, se, senao, enquanto, firme, quebrado, id, num_int, num_real, texto, +, -, *, /, \% =, ==, !=, >, >=, <, <=, \&\&, ||, !, (, ), \{, \}, ;, } \}$

$S = \text{Programa}$

$P$  (EBNF:  $\{ \}$  zero ou mais,  $[ ]$  opcional,  $( | )$  agrupamento, "x" terminal):

```
<Programa> ::= "gambiarra" <Bloco>
<Bloco> ::= "{" { <Instrucao> } "}"
<Instrucao> ::= <Declaracao> | <Atribuicao> | <Leitura> | <Escrita>
               | <Condicional> | <Repeticao>
<Declaracao> ::= "remendo" id "=" <Expressao> ";";
<Atribuicao> ::= id "=" <Expressao> ";";
<Leitura> ::= "pega" "(" id ")" ";";
<Escrita> ::= "mostra" "(" <Item> { "," <Item> } ")" ";";
<Item> ::= texto | <Expressao>
<Condicional> ::= "se" "(" <Expressao> ")" <Bloco> [ "senao" <Bloco> ]
<Repeticao> ::= "enquanto" "(" <Expressao> ")" <Bloco>
<Expressao> ::= <ExprE> { "||" <ExprE> }
<ExprE> ::= <ExprRel> { "&&" <ExprRel> }
<ExprRel> ::= <ExprAdit> [ <OpRel> <ExprAdit> ]
<OpRel> ::= "==" | "!=" | ">" | ">=" | "<" | "<="
<ExprAdit> ::= <Termo> { ( "+" | "-" ) <Termo> }
<Termo> ::= <Unario> { ( "*" | "/" | "%" ) <Unario> }
<Unario> ::= ( "!" | "-" ) <Unario> | <Fator>
<Fator> ::= num_int | num_real | "firme" | "quebrado" | id | "(" <Expressao> ")"
```

### 4.1 Precedência, associatividade e ausência de ambiguidade

Cada nível de precedência é uma variável da gramática: quanto mais fraco o operador, mais perto da raiz fica a regra dele. Os parênteses (em  $\langle \text{Fator} \rangle$ ) são o único jeito de "furar" os níveis.

| Nível          | Operadores                                                                      | Variável  | Associatividade                              |
|----------------|---------------------------------------------------------------------------------|-----------|----------------------------------------------|
| 1 (mais fraco) | <code>  </code>                                                                 | Expressao | esquerda                                     |
| 2              | <code>&amp;&amp;</code>                                                         | ExprE     | esquerda                                     |
| 3              | <code>== != &gt; &gt;= &lt; &lt;=</code>                                        | ExprRel   | não associativo (a < b < c é erro sintático) |
| 4              | <code>+ -</code>                                                                | ExprAdit  | esquerda (10 - 3 - 2 = 5)                    |
| 5              | <code>* / %</code>                                                              | Termo     | esquerda                                     |
| 6              | <code>!</code> e - unários                                                      | Unario    | direita (- -3 = 3)                           |
| 7 (mais forte) | número, variável, <code>firme</code> , <code>quebrado</code> , <code>( )</code> | Fator     | —                                            |

É a mesma ordem do C, por isso o C gerado se comporta igual. **Sem recursão à esquerda:** a forma  $A \rightarrow A \text{ op } B$  (que associa à esquerda, mas faria a descida recursiva chamar a si mesma sem consumir token) foi escrita como  $A \rightarrow B \{ \text{op } B \}$ , que no parser é um `while` que junta cada termo novo à esquerda da árvore já montada. **LL(1):** as alternativas de  $\langle \text{Instrucao} \rangle$  começam com tokens diferentes (`remendo`, `id`, `pega`, `mostra`, `se`, `enquanto`), e em  $\langle \text{Item} \rangle$  o token `texto` não pertence a PRIMEIROS( $\langle \text{Expressao} \rangle$ ). Por isso basta espiar um token para escolher a regra.

## 4.2 Árvore de derivação e AST de $2 + 3 * 4$

Programa `exemplos/precedencia.gam`, saída real de `python transpilador.py exemplos/precedencia.gam --derivacao --ast --executar`:

```
// Exemplo curto para mostrar a precedencia (relatorio e demo).
gambiarra {
    remendo r = 2 + 3 * 4;
    mostra(r);
}
```

===== ÁRVORE DE DERIVAÇÃO =====

```
<Programa>
├─ gambiarra
├─ <Bloco>
│   └─ {
│       └─ <Instrucao>
│           └─ <Declaracao>
│               └─ remendo
│                   └─ id (r)
│                       └─ =
│                           └─ <Expressao>
│                               └─ <ExprE>
│                                   └─ <ExprRel>
│                                       └─ <ExprAdit>
│                                           └─ <Termo>
│                                               └─ <Unario>
│                                                   └─ <Fator>
│                                                       └─ num_int (2)
│                                           └─ +
│                                               └─ <Termo>
│                                                   └─ <Unario>
│                                                       └─ <Fator>
│                                                           └─ num_int (3)
│                                           └─ *
│                                               └─ <Unario>
│                                                   └─ <Fator>
│                                                       └─ num_int (4)
│                           └─ ;
│           └─ <Instrucao>
│               └─ <Escrita>
│                   └─ mostra
│                       └─ (
│                           └─ <Item>
│                               └─ <Expressao>
│                                   └─ <ExprE>
│                                       └─ <ExprRel>
│                                           └─ <ExprAdit>
│                                               └─ <Termo>
│                                                   └─ <Unario>
│                                                       └─ <Fator>
│                                                           └─ id (r)
│                           └─ )
│                   └─ ;
│       └─ }
└─ FIM (fim do arquivo)
```

===== AST =====

```
Programa
├─ Declaracao(r = (2 + (3 * 4)))
└─ Print(r)
```

OK: sem erros. C gerado em `gerados/precedencia.c`

```
===== EXECUÇÃO =====  
14
```

O `*` ficou dentro de um `<Termo>`, mais fundo que o `+`, então é calculado primeiro:  $2 + (3 * 4) = 14$ . A AST guarda só a estrutura  $((2 + (3 * 4)))$ , sem `;`, chaves ou variáveis intermediárias. A precedência está na própria árvore, e o gerador não "conserta" nada.

## 5. Analisador sintático e AST

`parser.py` implementa um parser por **descida recursiva**: cada regra da EBNF é uma função (`programa()`, `bloco()`, `declaracao()`, ..., `fator()`). Na EBNF, `<Nome>` vira uma chamada de função, `{ }` vira `while`, `[ ]` vira `if` e um terminal vira `consume(TIPO)`. As duas ferramentas básicas são `proximo()`, que espia o tipo do token atual, e `consume(tipo)`, que confere o token e avança. Cada função devolve um nó da AST (`ast_nos.py`: `Declaracao`, `Atribuicao`, `Leitura`, `Escrita`, `Se`, `Enquanto`, `Binario`, `Unario`, literais e `Variavel`), com linha e coluna. O parser também registra as regras chamadas e os tokens consumidos, para imprimir a árvore de derivação (`--derivacao`).

**Erro sintático:** o parser para no primeiro erro e informa linha, coluna, token encontrado e o que era esperado. Um `;` esquecido é apontado no fim da linha onde ele falta, e não no token da linha seguinte.

## 6. Análise semântica

`semantico.py` percorre a AST **depois** do parser. Ele mantém a **tabela de símbolos** (nome → tipo, linha da declaração, nome no C) como uma pilha de escopos: cada bloco empilha um escopo ao entrar e o desempilha ao sair. Declarar olha só o topo da pilha; procurar vai do topo para baixo. Ele calcula o tipo de cada expressão de baixo para cima (literal pelo formato, variável pela tabela, operação pelas regras abaixo) e **lista todos os erros**. Com qualquer erro, o C não é gerado.

| Regra | Descrição                                                                                                                                                                                                                                                             | Exemplo de violação                                             |
|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| R1    | variável só pode ser usada depois de declarada                                                                                                                                                                                                                        | <code>mostra(pontos);</code> sem declarar                       |
| R2    | não pode declarar duas vezes no mesmo escopo                                                                                                                                                                                                                          | <code>remendo x = 1; remendo x = 2;</code>                      |
| R3    | escopo por bloco: <code>se</code> , <code>senao</code> e <code>enquanto</code> têm o seu                                                                                                                                                                              | usar fora do <code>se</code> uma variável declarada dentro dele |
| R4    | sombreamento permitido: um bloco interno pode redeclarar um nome de fora                                                                                                                                                                                              | — (é válido)                                                    |
| R5    | <code>+</code> <code>-</code> <code>*</code> <code>/</code> só com números; inteiro com inteiro dá inteiro; com um real, dá real. <code>%</code> só com inteiros                                                                                                      | <code>peso % 2</code> com <code>peso</code> real                |
| R6    | <code>&gt;</code> <code>&gt;=</code> <code>&lt;</code> <code>&lt;=</code> só com números; <code>==</code> <code>!=</code> entre números ou entre lógicos; <code>&amp;&amp;</code> <code>  </code> <code>!</code> só com lógicos; <code>-</code> unário só com números | <code>ligado + 1</code>                                         |
| R7    | atribuição: real ← inteiro vale (promoção); inteiro ← real não; número ↔ lógico não                                                                                                                                                                                   | <code>vidas = 2.5;</code> com <code>vidas</code> inteiro        |
| R8    | <code>/</code> entre inteiros é divisão inteira ( $7 / 2 = 3$ ), como no C                                                                                                                                                                                            | —                                                               |
| R9    | na declaração, o valor é conferido antes de o nome existir                                                                                                                                                                                                            | <code>remendo x = x + 1;</code> sem outro <code>x</code>        |
| R10   | a condição de <code>se/enquanto</code> tem que ser lógica                                                                                                                                                                                                             | <code>se (vidas) { }</code>                                     |

|     |                                                             |                                 |
|-----|-------------------------------------------------------------|---------------------------------|
| R11 | pega só lê variável inteira ou real, já declarada           | pega(ligado); com ligado lógico |
| R12 | a variável fica com o tipo do valor inicial, e ele não muda | (é o que dá sentido à R7)       |

**Por que separar da sintaxe:** mostra(x); e mostra(y); viram a mesma sequência de tokens (mostra ( id ) ;). O parser não tem como saber se y foi declarada; quem sabe é a tabela de símbolos. "Declarar antes de usar" depende de contexto.

## 7. Geração de código C

gerador.py só roda se léxico, parser e semântica passaram. Ele **percorre a AST** (não faz substituição de texto): cada tipo de nó tem uma função que escreve o C equivalente. Os tipos viram inteiro → int, real → float (o scanf usa %f) e lógico → int (0/1). As variáveis ganham o prefixo v\_, para um nome como int não quebrar o C. Toda operação ganha parênteses. mostra vira um único printf: %d para inteiro, %g para real, %s para texto e para lógico (impresso como firme/quebrado), com \n no fim. pega vira scanf com %d ou %f.

**Sombreamento:** dentro de um bloco, remendo x = x + 1; deve ler o x de fora (R9). No C, int v\_x = v\_x + 1; leria o x novo, ainda sem valor. Por isso a semântica dá à variável de dentro outro nome (v\_x\_2), e o comportamento é preservado (teste 07).

### 7.1 Exemplo completo: testes/02\_valido\_completo.gam

```
// Teste 02: condicional, repetição, entrada/saída e precedência (enunciado §8).
// A entrada (o que o 'pega' lê) está em 02_valido_completo.entrada.txt: 4
gambiarra {
    remendo limite = 0;
    mostra("Quantos remendos?");
    pega(limite);

    // precedência: * antes de +, e os parênteses mudam a ordem
    remendo a = 2 + 3 * 4;           // 14
    remendo b = (2 + 3) * 4;         // 20
    remendo c = 10 - 3 - 2;          // 5 (associatividade à esquerda)
    remendo d = 7 / 2;               // 3 (divisão inteira, regra R8)
    remendo e = 7 % 2;               // 1
    remendo media = (a + b) / 4.0;    // 8.5 (inteiro com real dá real, regra R5)
    mostra("a = ", a, ", b = ", b, ", c = ", c);
    mostra("d = ", d, ", e = ", e, ", media = ", media);

    remendo contador = 1;
    remendo soma = 0;
    enquanto (contador <= limite) {
        soma = soma + contador;
        mostra("remendo ", contador, " aplicado");
        contador = contador + 1;
    }
    mostra("soma = ", soma);

    remendo firmeza = soma > 5 && !(limite == 0);
    se (firmeza || quebrado) {
        mostra("A gambiarra esta firme: ", firmeza);
    } senao {
        mostra("A gambiarra quebrou");
    }
}
/* Saída esperada do programa (com a entrada 4):
```

```
Quantos remendos?
a = 14, b = 20, c = 5
d = 3, e = 1, media = 8.5
remendo 1 aplicado
remendo 2 aplicado
remendo 3 aplicado
remendo 4 aplicado
soma = 10
A gambiarra esta firme: firme
*/
```

C gerado (gerados/02\_valido\_completo.c):

```
#include <stdio.h>

int main(void) {
    int v_limite = 0;
    printf("%s\n", "Quantos remendos?");
    scanf("%d", &v_limite);
    int v_a = 2 + (3 * 4);
    int v_b = (2 + 3) * 4;
    int v_c = (10 - 3) - 2;
    int v_d = 7 / 2;
    int v_e = 7 % 2;
    float v_media = (v_a + v_b) / 4.0f;
    printf("%s%d%s%d%s%d\n", "a = ", v_a, ", b = ", v_b, ", c = ", v_c);
    printf("%s%d%s%d%s%g\n", "d = ", v_d, ", e = ", v_e, ", media = ", v_media);
    int v_contador = 1;
    int v_soma = 0;
    while (v_contador <= v_limite) {
        v_soma = v_soma + v_contador;
        printf("%s%d%s\n", "remendo ", v_contador, " aplicado");
        v_contador = v_contador + 1;
    }
    printf("%s%d\n", "soma = ", v_soma);
    int v_firmeza = (v_soma > 5) && !(v_limite == 0);
    if (v_firmeza || 0) {
        printf("%s%s\n", "A gambiarra esta firme: ", (v_firmeza ? "firme" : "quebrado"));
    } else {
        printf("%s\n", "A gambiarra quebrou");
    }
    return 0;
}
```

Execução (entrada do teclado: 4):

OK: sem erros. C gerado em gerados/02\_valido\_completo.c

```
===== EXECUÇÃO =====
Quantos remendos?
a = 14, b = 20, c = 5
d = 3, e = 1, media = 8.5
remendo 1 aplicado
remendo 2 aplicado
remendo 3 aplicado
remendo 4 aplicado
soma = 10
A gambiarra esta firme: firme
```

## 8. Tratamento de erros

| Categoria | Quando                                                                                        | Resposta                                                                 |
|-----------|-----------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| Léxico    | símbolo fora de $\Sigma$ , número ou identificador malformado, texto ou comentário sem fechar | linha, coluna e lexema; continua lendo e lista todos; não chama o parser |
| Sintático | tokens válidos numa ordem que a gramática não gera                                            | linha, coluna, token encontrado e esperado; para no primeiro             |
| Semântico | violação de R1 a R12                                                                          | linha, coluna, nome da variável e regra violada; lista todos; não gera C |

### 8.1 Erro léxico — testes/03\_erro\_lexico.gam

```
// Teste 03: erros léxicos (enunciado §7 e EI03 P4b). O léxico reporta TODOS e continua lendo.
gambiarra {
    remendo x = 10;
    mostra(x @ 2);
    remendo 2abc = 5;
    remendo y = 3.4.5;
    remendo preço = 1;
    x = x & 1;
    mostra("texto sem fechar");
    remendo z = 5.;
}
/* comentário de bloco que nunca fecha
```

```
===== ERROS LÉXICOS =====
Erro léxico [linha 4, coluna 14]: caractere que não pertence à linguagem: '@'
Erro léxico [linha 5, coluna 13]: número malformado (inteiro: 10; real: 10.5; nome não começa com
dígito): '2abc'
Erro léxico [linha 6, coluna 17]: número malformado (inteiro: 10; real: 10.5; nome não começa com
dígito): '3.4.5'
Erro léxico [linha 7, coluna 13]: identificador com caractere inválido (use só letras sem acento,
dígitos e _): 'preço'
Erro léxico [linha 8, coluna 11]: caractere que não pertence à linguagem: '&'
Erro léxico [linha 9, coluna 12]: texto sem fechar aspas (o texto não pode quebrar linha): '"texto
sem fechar)';
Erro léxico [linha 10, coluna 17]: número malformado (inteiro: 10; real: 10.5; nome não começa com
dígito): '5.'
Erro léxico [linha 12, coluna 1]: comentário de bloco '/*' aberto e nunca fechado com '*/': '/*
comentário de bloco que nun...'

8 erro(s) léxico(s). Processamento interrompido.
```

### 8.2 Erros sintáticos — 04\_erro\_sintatico.gam e 04b\_erro\_sintatico\_expressao.gam

```
// Teste 04: erro sintático (enunciado §8). Falta o ';' no fim da linha 4.
gambiarra {
    remendo x = 10;
    remendo y = x * 2
    mostra(y);
}
```

```
===== ERRO SINTÁTICO =====
Erro sintático [linha 4, coluna 22]: esperava ';' depois de '2' (fim da linha 4), mas encontrei
'mostra' na linha 5

Processamento interrompido.
```

```
// Teste 04b: erro sintático do enunciado §7: declaração termina sem expressão após '='.
gambiarra {
    remendo total = ;
    mostra(total);
}
```

===== ERRO SINTÁTICO =====

Erro sintático [linha 3, coluna 21]: esperava um valor (número, variável, firme, quebrado ou '('), mas encontrei ';'.

Processamento interrompido.

### 8.3 Erros semânticos — testes/05\_erro\_semantico.gam

```
// Teste 05: erros semânticos (enunciado §5 e §8). A semântica lista TODOS e não gera o C.
gambiarra {
    remendo vidas = 3;
    remendo peso = 2.5;
    remendo ligado = firme;

    mostra(pontos);           // R1: usada sem declarar
    total = 10;               // R1: atribuição sem declarar
    remendo vidas = 5;        // R2: declarada duas vezes no mesmo escopo
    vidas = peso;              // R7: inteiro não recebe real
    ligado = 1;                // R7: lógico não recebe número
    remendo resto = peso % 2;  // R5: % só entre inteiros
    se (vidas) {               // R10: condição tem que ser lógica
        mostra("nunca");
    }
    enquanto (ligado + 1 > 0) { // R5: lógico não entra em conta
        pega(ligado);         // R11: pega não lê lógico
    }
    remendo x = x + 1;         // R9: o x ainda não existe
}
```

===== ERROS SEMÂNTICOS =====

Erro semântico [linha 7, coluna 12]: a variável 'pontos' foi usada antes de ser declarada (regra R1)

Erro semântico [linha 8, coluna 5]: atribuição à variável 'total', que não foi declarada (regra R1: declare antes com 'remendo total = ...;')

Erro semântico [linha 9, coluna 13]: a variável 'vidas' já foi declarada neste escopo, na linha 3 (regra R2: não pode declarar duas vezes no mesmo escopo)

Erro semântico [linha 10, coluna 5]: a variável 'vidas' é inteiro e não pode receber um valor real (regra R7)

Erro semântico [linha 11, coluna 5]: a variável 'ligado' é lógico e não pode receber um valor inteiro (regra R7)

Erro semântico [linha 12, coluna 26]: o operador '%' só aceita inteiros, mas recebeu real e inteiro (regra R5)

Erro semântico [linha 13, coluna 9]: a condição do 'se' tem que ser lógica, mas é inteiro (regra R10; ex.: use 'x > 0' em vez de 'x')

Erro semântico [linha 16, coluna 22]: o operador '+' só aceita números, mas recebeu lógico e inteiro (regra R5)

Erro semântico [linha 17, coluna 14]: pega(ligado): a variável 'ligado' é lógica; pega só lê inteiro ou real (regra R11)

Erro semântico [linha 19, coluna 17]: a variável 'x' foi usada antes de ser declarada (regra R1)

10 erro(s) semântico(s). O código C NÃO foi gerado.

## 9. Suíte de testes

| Arquivo | Objetivo |
|---------|----------|
|---------|----------|

|                             |                                                                                                                                                                      |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 01_valido_basico            | declaração, atribuição e saída                                                                                                                                       |
| 02_valido_completo          | se/senao, enquanto, pega/mostra, precedência <code>2 + 3 * 4</code> e <code>(2 + 3) * 4</code> , associatividade <code>10 - 3 - 2</code> , divisão inteira, promoção |
| 03_erro_lexico              | @, 2abc, 3.4.5, preço, &, 5., texto e comentário sem fechar                                                                                                          |
| 04_erro_sintatico / 04b     | ; esquecido / declaração sem expressão depois do = (exemplo do enunciado, §7)                                                                                        |
| 05_erro_semantico           | R1, R2, R5, R7, R9, R10 e R11 no mesmo arquivo                                                                                                                       |
| 06_lexico_todos_tokens      | todos os tipos de token válidos (só o léxico)                                                                                                                        |
| 07_valido_escopo            | escopo por bloco e sombreadamento                                                                                                                                    |
| 08_valido_exemplo_enunciado | o exemplo teste.bir1 do enunciado, reescrito em Gambiarra                                                                                                            |
| 09_valido_com_bom           | arquivo salvo com BOM (Bloco de Notas/PowerShell)                                                                                                                    |

Cada teste tem a saída esperada em `.esperado.txt`. `python rodar_testes.py` roda o transpilador em cada arquivo (nos válidos, também compila com `gcc -Wall` e executa) e compara com a saída esperada. Resultado no momento da geração deste relatório:

```
OK      01_valido_basico.gam
OK      02_valido_completo.gam
OK      03_erro_lexico.gam
OK      04_erro_sintatico.gam
OK      04b_erro_sintatico_expressao.gam
OK      05_erro_semantico.gam
OK      06_lexico_todos_tokens.gam
OK      07_valido_escopo.gam
OK      08_valido_exemplo_enunciado.gam
OK      09_valido_com_bom.gam

10 de 10 testes passaram.
```

O C gerado de cada teste válido fica em `gerados/`.

## 10. Como executar

Pré-requisitos: Python 3.10+ (só a biblioteca padrão) e gcc.

```
python transpilador.py programa.gam          (gera gerados/programa.c)
python transpilador.py programa.gam --executar (gera, compila com gcc e roda)
python transpilador.py programa.gam --tokens --derivacao --ast --c
python rodar_testes.py                      (suíte de testes)
```

Organização: `tokens.py` e `erros.py` (base), `lexico.py` (1), `parser.py` + `ast_nos.py` (2), `semantico.py` (3), `gerador.py` (4) e `transpilador.py` (liga as estações). Detalhes no `README.md`.

## 11. Uso de ferramentas de Inteligência Artificial

| Item       | Declaração                                          |
|------------|-----------------------------------------------------|
| Ferramenta | Claude Code (modelo Claude Opus 5.5, da Anthropic). |

|                             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Finalidades                 | (1) busca de material e explicação dos conceitos da disciplina (léxico, GLC, BNF/EBNF, derivação, precedência, recursão à esquerda, descida recursiva, AST, semântica, geração de código); (2) organização do planejamento e levantamento dos requisitos do enunciado; (3) levantamento das decisões de projeto, apresentadas como perguntas com opções: <b>cada decisão foi escolhida pelo grupo</b> ; (4) <b>escrita do código do transpilador</b> , dos testes e dos rascunhos da documentação, a partir dessas decisões; (5) depuração. |
| Etapas                      | estudo, especificação (léxico e gramática), implementação (léxico, parser, AST, semântica e geração), testes e documentação.                                                                                                                                                                                                                                                                                                                                                                                                                |
| Como verificamos            | suíte com 10 testes (saídas conferidas à mão antes de virarem gabarito); o C gerado compila com <code>gcc -Wall</code> sem avisos e produz a saída esperada; testes extras de casos-limite (programa vazio, escopo, relacional encadeado, bloco sem fechar, conteúdo depois do fim, texto em expressão). Durante os testes foram achados e corrigidos dois problemas: arquivos com BOM eram rejeitados pelo léxico (teste 09) e a última folha da árvore de derivação saía vazia.                                                           |
| Fontes de validação         | enunciado do TP1 (v6) e textos do Classroom; slides das Aulas 02 e 03; documentação do módulo <code>re</code> do Python. <b>[PREENCHER: capítulos dos livros efetivamente consultados]</b>                                                                                                                                                                                                                                                                                                                                                  |
| O que o grupo fez / alterou | escolha do tema, do vocabulário (vindo de um integrante) e de todas as decisões de projeto (tipos, sintaxe, precedência, regras semânticas, formato da saída); revisão das saídas dos testes. <b>[PREENCHER: alterações feitas pelo grupo no código e no texto depois do estudo]</b>                                                                                                                                                                                                                                                        |

O grupo é responsável por todo o conteúdo entregue, e todos os integrantes devem saber explicar cada parte.

## 12. Referências

AHO, Alfred V.; SETHI, Ravi; ULLMAN, Jeffrey D. *Compiladores: princípios, técnicas e ferramentas*. 2. ed. São Paulo: Pearson Addison-Wesley, 2007.

MENEZES, Paulo Blauth. *Linguagens formais e autômatos*. 6. ed. Porto Alegre: Bookman, 2011.

PRICE, Ana Maria de Alencar; TOSCANI, Simão Sirineo. *Implementação de linguagens de programação: compiladores*. 2. ed. Porto Alegre: Bookman, 2001.

BIRL — Bambam's "It's Show Time" Recursive Language. Disponível em: <https://birl-language.github.io/>.

CAVALHEIRO, Marcos R. M. *Trabalho Prático 1 — Projeto e Implementação de uma Linguagem de Programação Temática* (v6) e slides das Aulas 02 e 03. UNIJUÍ, 2026.

PYTHON SOFTWARE FOUNDATION. *re — Regular expression operations*. Disponível em: <https://docs.python.org/3/library/re.html>.